Processamento de Linguagem Natural

Luís Filipe Cunha
lfc@di.uminho.pt

José João Almeida
jj@di.uminho.pt

Natural Language Processing Tools

NLP Libraries for Text Processing

- Tokenization
- Linguistic analysis
- Information extraction

Why Do We Need NLP Tools?

Natural language processing involves **many common tasks**, such as:

- Tokenization
- Sentence segmentation
- Part-of-speech tagging
  Lemmatization
- Named entity recognition
  Dependency parsing

Instead of implementing everything from scratch, we use **NLP libraries**.

Benefits:

- Faster development
- Reproducible pipelines
- Access to pretrained models
- Standardized processing

Popular NLP Libraries

Several libraries are commonly used for NLP.

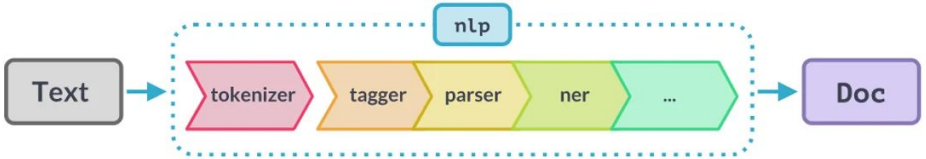
Main examples:

- **NLTK**: Teaching, learning NLP concepts
- **spaCy**: Production systems, industrial NLP
- **Stanza**: Academic research, linguistic analysis
- **Stanford CoreNLP** (java): Academic research, linguistic analysis
- **Apache OpenNLP:** Academic research, industry
- **Gensim**: Research on semantic models and embeddings
- **Transformers**: Deep learning and modern neural NLP

spaCy Features

Tokenization	Segmenting text into words, punctuations marks etc.
Part-of-speech (POS) **Tagging**	Assigning word types to tokens, like verb or noun.
Dependency Parsing	Assigning syntactic dependency labels, describing the relations between individual tokens, like subject or object.
Lemmatization	Assigning the base forms of words. For example, the lemma of “was” is “be”, and the lemma of “rats” is “rat”.
Sentence Boundary Detection (SBD)	Finding and segmenting individual sentences.
Named Entity Recognition (NER)	Labelling named “real-world” objects, like persons, companies or locations.
Entity Linking (EL)	Disambiguating textual entities to unique identifiers in a knowledge base.
Similarity	Comparing words, text spans and documents and how similar they are to each other.
Text Classification	Assigning categories or labels to a whole document, or parts of a document.
Rule-based Matching	Finding sequences of tokens based on their texts and linguistic annotations, similar to regular expressions.
Training	Updating and improving a statistical model’s predictions.

spaCy Pipeline



NAME	COMPONENT	CREATES	DESCRIPTION
tokenizer PROCESSING PIPELINE	`Tokenizer`	`Doc`	Segment text into tokens.
tagger	`Tagger`	`Token.tag`	Assign part-of-speech tags.
parser	`DependencyParser`	`Token.head`, `Token.dep`, `Doc.sents`, `Doc.noun_chunks`	Assign dependency labels.
ner	`EntityRecognizer`	`Doc.ents`, `Token.ent_iob`, `Token.ent_type`	Detect and label named entities.
lemmatizer	`Lemmatizer`	`Token.lemma`	Assign base forms.
textcat	`TextCategorizer`	`Doc.cats`	Assign document labels.
custom	custom components	`Doc._.xxx`, `Token._.xxx`, `Span._.xxx`	Assign custom attributes, methods or properties.

Tokenization

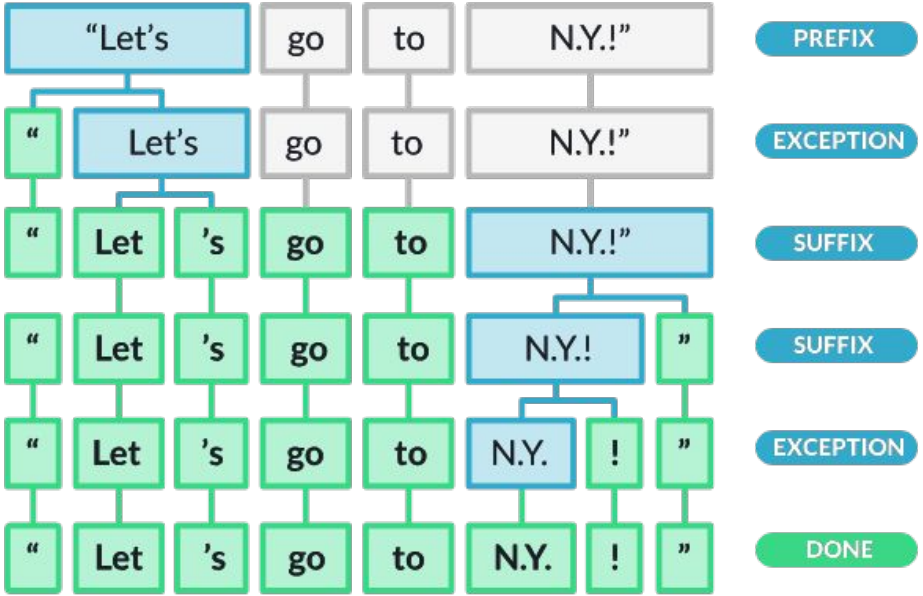
During processing, spaCy first tokenizes the text, i.e. segments it into words, punctuation and so on. This is done by applying rules specific to each language.

Tokenizer exception: Special-case rule to split a string into several tokens or prevent a token from being split when punctuation rules are applied.

Prefix: Character(s) at the beginning, e.g. $, (, “

Suffix: Character(s) at the end, e.g. km,), ”, !.

Infix: Character(s) in between, e.g. -, --, /, …



Part-of-speech tags and dependencies

Lemma: The base form of the word.
run, runs, ran, running → **run**

POS: grammatical category of a word in a sentence (Noun, Verb, Adj, Det…)

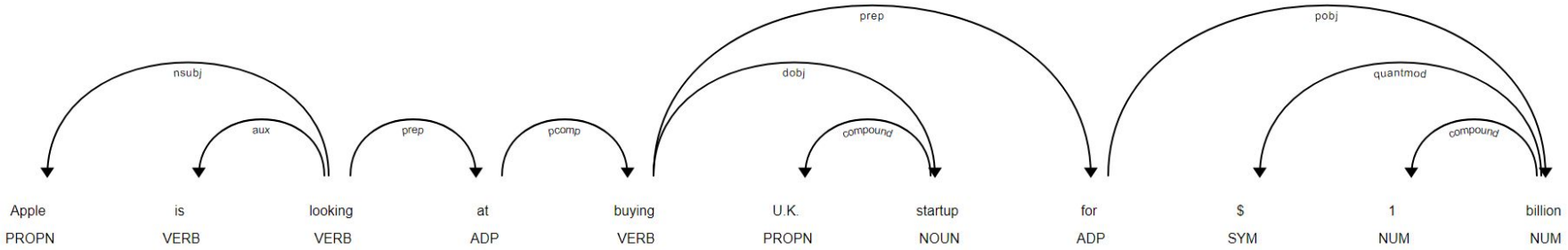
Dep: Syntactic dependency. Analyses the grammatical structure of a sentence by identifying syntactic relationships between words.

is stop: very common word in a language that usually carries little semantic meaning by itself

TEXT	LEMMA	POS	TAG	DEP	SHAPE	ALPHA	STOP
Apple	apple	PROPN	NNP	nsubj	Xxxxx	True	False
is	be	AUX	VBZ	aux	xx	True	True
looking	look	VERB	VBG	ROOT	xxxx	True	False
at	at	ADP	IN	prep	xx	True	True
buying	buy	VERB	VBG	pcomp	xxxx	True	False
U.K.	u.k.	PROPN	NNP	compound	X.X.	False	False
startup	startup	NOUN	NN	dobj	xxxx	True	False
for	for	ADP	IN	prep	xxx	True	True
$	$	SYM	$	quantmod	$	False	False
1	1	NUM	CD	compound	d	False	False
billion	billion	NUM	CD	pobj	xxxx	True	False

Dpendency Parser Tree

```
from spacy import displacy
displacy.serve(doc, style="dep") # ner
```



Named Entities

Embaixador de Portugal Profissao , em Londres Local , no Rio de Janeiro Local , Brasil Local , entre outros . Contém a carta do Embaixador Profissao , António de Faria Pessoa , Junto da Santa Sé Organizacao , em Roma Local , dirigida ao Dr. Salazar Pessoa , a informar que se deslocou ao Arquivo Secreto do Vaticano Organizacao e pôde observar " numa das salas decoradas com frescos do tempo do Papa Profissao paulo V Pessoa nos começos do séc . XVII a Data pintura representando a entrega a D. Afonso Henriques Pessoa da coroa real pelo Papa Profissao Alexandre Pessoa III (...) " . Entre outros .

Rule-Based Pattern Matching

Matchers help you find and extract information from text by defining patterns based on linguistic features such as:

- POS,
- TAG,
- MORPH,
- DEP,
- LEMMA,
- SHAPE
- IS_PUNCT,
- IS_SPACE,
- IS_STOP
- ENT_TYPE

...

A pattern added to the `Matcher` consists of a list of dictionaries. Each dictionary describes one token and its attributes.

```
from spacy.matcher import Matcher
matcher = Matcher(nlp.vocab)

pattern = [
  {"LOWER": "i"},
  {"LEMMA": {"IN": ["like", "love"]}},
  {"POS": "NOUN", "OP": "+"}
]
matcher.add("match_id", [pattern])
matches = matcher(doc)

#[(match_id, start, end)]
```

https://spacy.io/api/matcher

Entity Ruler

You can use token attributes like:

- TEXT
- LOWER
- LEMMA
- POS
- TAG
- DEP
- SHAPE
- IS_ALPHA
- IS_DIGIT
- IS_STOP

...

```
ruler = nlp_ner.add_pipe("entity_ruler", last=True) # before, after, first

patterns = [
    {"label": "Pessoa", "pattern": "Dumbledore"},
    {
        "label": "Pessoa",
        "pattern": [{"LOWER": "albus"}, {"LOWER": "dumbledore"}]
    },
    {
    "label": "EMAIL",
    "pattern": [{"TEXT": {"REGEX": "^[\\w.-]+@[\\w.-]+\\.[a-zA-Z]{2,}$"}}]
    }
]
ruler.add_patterns(patterns)
```

spaCy Installation

```
pip install -U pip setuptools wheel
pip install -U spacy
python -m spacy download pt_core_news_sm
```

https://spacy.io/usage

```
import spacy
nlp = spacy.load("pt_core_news_sm")

doc = nlp("A Maria é aluna da Universidade do
Minho que se situa em Braga, Portugal.")
```

Exercises - Information Extraction

1- Given a news document text, extract all the entities of type PERSON and ORG present in that text. Convert the output to JSON.

2 - Given a text, count the occurrences of verbs. Next, calculate the occurrence of verbs ignoring their conjugation forms. That is, different forms of the same verb should be considered as the same verb. What are are the top-10 verbs used in the corpus?

3- Triplet Extraction: Use spaCy to extract relevant triplets from the text. For example, a Person followed by a verb and a noun. You can also use the nsub and obj from the syntatic dependency tree.

Harry Friends!

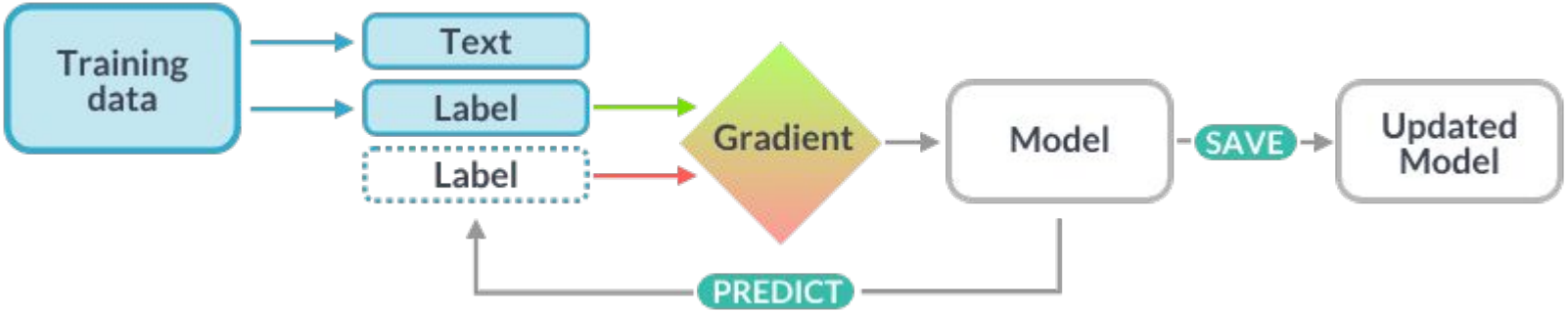
Using spaCy, extract all **Characters** from the text and compute their relationships based on sentence co-occurrence.

📌 Two characters are considered friends if they appear in the **same sentence**.

Output Format

```
{
  "Harry": {
      "Ron": 152,
      "Hermione": 148,
      "Hagrid": 63,
      "Dumbledore": 41,
      "Ginny": 37,
      "Neville": 29,
      "Draco": 26,
      "Snape": 18,
      "McGonagall": 15
  }
}
```

Train a model




```
spacy convert -c iob train_iob.iob ./datasets
spacy init config config.cfg --pipeline ner --lang pt
spacy train config.cfg --output ./outputs
```

CoNLL-U Format

https://universaldependencies.org/format.html

Columns: Token, lema, pos, tag, morph, head, dep, entity

```
# sent_id = 1
# text = They buy and sell books.
1   They     they    PRON    PRP    Case=Nom|Number=Plur                2   nsubj   2:nsubj|4:nsubj   _
2   buy      buy     VERB    VBP    Number=Plur|Person=3|Tense=Pres     0   root    0:root            _
3   and      and     CCONJ   CC     _                                   4   cc      4:cc              _
4   sell     sell    VERB    VBP    Number=Plur|Person=3|Tense=Pres     2   conj    0:root|2:conj     _
5   books    book    NOUN    NNS    Number=Plur                         2   obj     2:obj|4:obj       SpaceAfter=No
6   .        .       PUNCT   .      _                                   2   punct   2:punct           _

# sent_id = 2
# text = I have no clue.
1   I       I       PRON    PRP   Case=Nom|Number=Sing|Person=1     2   nsubj   _   _
2   have    have    VERB    VBP   Number=Sing|Person=1|Tense=Pres   0   root    _   _
3   no      no      DET     DT    PronType=Neg                      4   det     _   _
4   clue    clue    NOUN    NN    Number=Sing                       2   obj     _   SpaceAfter=No
5   .       .       PUNCT   .     _                                 2   punct   _   _
```

Processamento de Linguagem Natural

Luís Filipe Cunha
lfc@di.uminho.pt

José João Almeida
jj@di.uminho.pt